

Rehearse Everything, Remember Nothing: Attic-KV Rehearses What Will Be Read

Zhiyun Shi

Nanyang Technological University

Singapore

ZHIYUN004@e.ntu.edu.sg

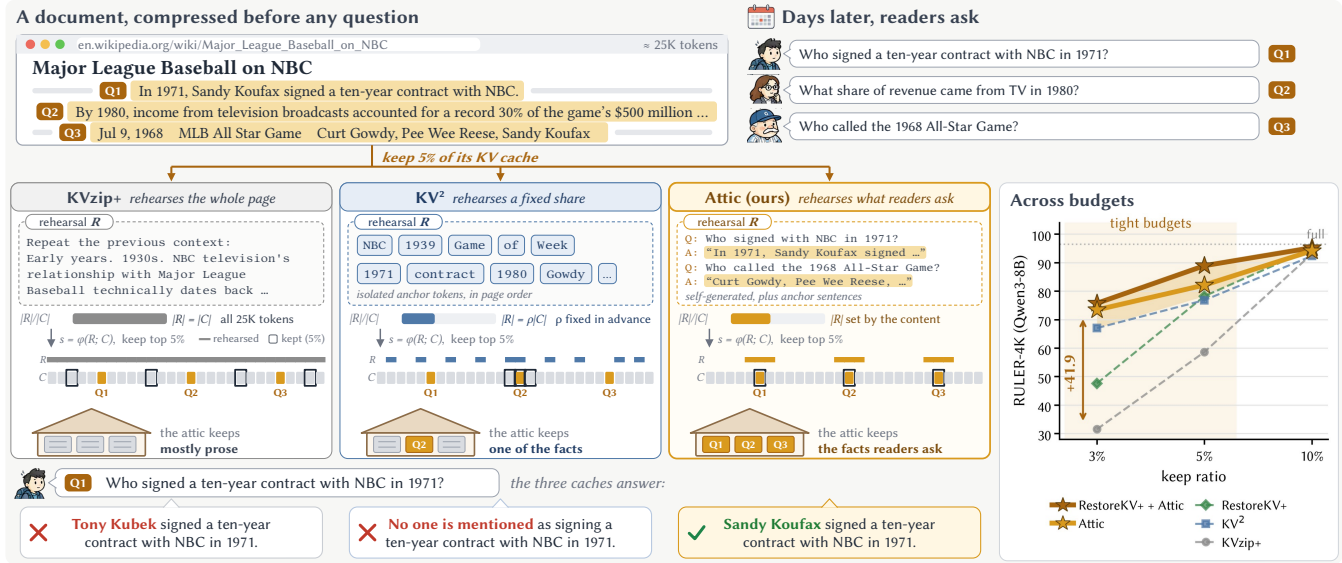


Figure 1: Rehearse everything, remember nothing. A document, here a Wikipedia page, is compressed once, to 5% of its KV cache, before any question arrives. Each method feeds a rehearsal R after the page and keeps the 5% of entries that R attends to most, so a compressed cache, like a small attic, keeps what it rehearses. KVzip+ rehearses the whole page and keeps mostly prose; KV² rehearses a fixed share of isolated anchors and keeps only some facts; ATTIC rehearses self-generated question-answer pairs and anchor sentences, as much as the page holds, and keeps the facts readers later ask about. Answers are verbatim outputs of Qwen3-8B on this LooGLE page. (Right) RULER-4K score versus keep ratio on Qwen3-8B: the gap opens at tight budgets and reaches 41.9 points over KVzip+ at 3%.

Abstract

Many key-value (KV) caches are compressed before anyone knows what will be asked of them: a document cached for retrieval, a prompt prefix shared across requests, the memory of a long conversation. The prevailing approach scores KV entries by *rehearsal*: the model rereads the context and keeps the entries it attends to, assuming that the more completely a cache rehearses its context, the better it remembers it. We show that under tight budgets this assumption backfires: **rehearse everything, remember nothing**. At a 3% keep ratio, rereading the whole context keeps 31.5 of 96.5 points on RULER, and on LongBench’s natural-text tasks it falls below methods that rehearse nothing at all. The cause is that **a cache keeps what it rehearses**: rereading spreads the budget across the whole context, so the answer’s own entries survive at little more than chance. Like a student before an exam, a cache remembers more by testing itself than by rereading. Two principles follow: **rehearse what will be read**, and **rehearse as much as there is**. We instantiate them as **ATTIC-KV** (ATTIC for short), a training-free rehearsal in which the model quizzes itself with question-answer

pairs that quote the context, alongside anchor tokens in a content-adaptive amount. Changing only the rehearsal lifts three hosts that score it in three different ways: ATTIC alone is the best training-free method in all eight settings we test on RULER and LongBench’s natural-text tasks, and plugged into the gradient-based KVgrad and the trained RestoreKV+, it raises them by up to 17.1 and 28.1 points. Its advantage grows as the budget shrinks, reaching 41.9 points over full rereading at a 3% keep ratio, and it compresses faster than rereading the whole context.

Keywords

KV cache compression, query-agnostic compression, long-context LLMs, retrieval-augmented generation

1 Introduction

When Watson learns that Sherlock Holmes does not know that the Earth travels round the Sun, Holmes explains that a man’s brain is like a little empty attic [7]. A fool stocks it with every sort of lumber he comes across, until the knowledge that might be

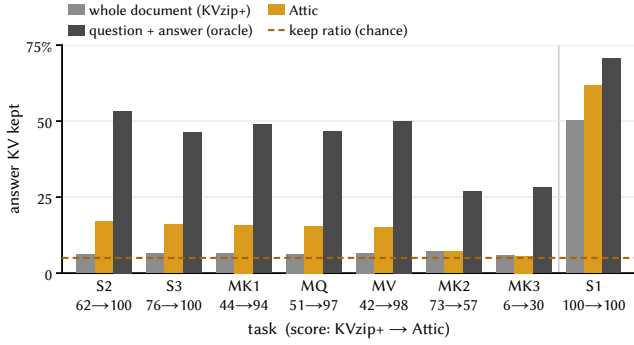


Figure 2: Rehearse everything, remember nothing. Share of each answer’s KV entries, over all layers and heads, that the cache keeps (Qwen3-8B, RULER-4K development split, the eight needle tasks, keep ratio 5%). Whole-document rehearsal keeps the answer no more often than any other token (dashed line); **ATTIC** keeps it on the tasks it answers, and an oracle that rehearses the gold answer shows how much a 5% cache can hold. Below each task: its score with whole-document rehearsal → with **ATTIC**.

useful is crowded out; a skilful workman takes in nothing but the tools that may help him in his work, and of those he keeps a large assortment. For, as Holmes warns, “it is a mistake to think that that little room has elastic walls.” A compressed key–value (KV) cache is such an attic: it holds a small, fixed fraction of a context’s keys and values, and what it holds decides which questions the model can still answer.

Many KV caches are filled before the questions are known. Retrieval-augmented and search systems precompute the caches of documents and reuse them across requests [12, 18, 34, 51]; serving systems share the cache of a long prompt prefix across users; assistants keep the cache of a long conversation for turns that have not happened yet. Each such cache is compressed once and then serves many queries that are unknown when it is compressed [23, 43]. Compression must therefore be *query-agnostic* and *one-shot*, and because memory grows with the number of caches, it pays off only when it is extreme: a 5% keep ratio fits 20 times as many caches in the same memory, a 30% ratio only about 3 times (Section 2).

The prevailing paradigm for this setting is **rehearsal scoring**, introduced by KVzip [20]. Before any question arrives, the model *rehearses* the context, rereading it while attending to the cache; each KV entry is scored by the attention it receives, and the top-scoring entries are kept. Its successors refine the scorer (KVzip+ and KVzap [17], Fast KVzip [19], EchoPress [29], KVgrad [22]) or what a retained subset can express (RestoreKV [1]), yet all of them rehearse the same target: the entire context. KVzip argues explicitly for full-context rehearsal [20], and the whole line shares its default assumption: the more completely a cache rehearses its context, the more faithfully it preserves it. KV² [47] rehearses only a fixed fraction of the context, its anchor tokens; but like every rehearsal before it, what it rehearses is a copy of the context.

We show that this assumption fails exactly where compression matters: **rehearse everything, remember nothing**. On

RULER-4K [15], Qwen3-8B scores 96.5 without compression, but full-context rehearsal (KVzip+) at a 3% keep ratio keeps only 31.5; on the natural-text tasks of LongBench it falls below methods that rehearse nothing at all (28.6 vs. up to 36.5). Where does the budget go? For each question, we measure how much of its answer’s KV survives compression: at a 5% keep ratio, full-context rehearsal keeps 5.7–7.1% of it on seven of eight needle tasks, no more than of any other token (Figure 2). The cause is that **a cache keeps what it rehearses** (Section 3): rehearsed positions cover about 5% of the context yet draw a fifth to a third of the retained budget, and when the model rehearses only a list of facts, its score on each task tracks the coverage of that list. Capacity is not the bottleneck: a 5% cache scored by rehearsing the answer loses almost nothing (93.5 vs. 94.7 uncompressed). A full-context rehearsal spends the budget on fluent prose that no query reads and crowds out the few facts that every query reads, and the tighter the budget, the harder the crowding.

The failure has a familiar shape. A student preparing for an exam can reread the chapter, reread only what they highlighted, or test themselves on it. A large review of study techniques rates the first two of low utility and the third of high utility [8]: practice that produces answers beats seeing the material again [42]. Existing rehearsals are the first two strategies: full-context rehearsal rereads the whole context, and KV² rereads its highlights. Neither rehearses what a reader will read out of the context, and a compressed cache, like a student, remembers what it practises. What counts as practice is specific: rehearsing a question without its answer keeps a cache that scores 47.0, against 93.5 when the answer is rehearsed with it, because attention lands on the answer only while the answer is produced (Section 4.1). Query-agnostic compression is thus first a question of how a cache should study: on Qwen3-8B RULER-4K, replacing the KVzip+ scorer with KVgrad’s gradient attribution adds 13.4 points at a 5% keep ratio and 22.0 at 3%, while replacing its rehearsal with ours adds 23.5 and 41.9.

Two principles follow. **Rehearse what will be read**: a cache should rehearse what future queries will read out, not the whole context; rehearsing only the facts that can be asked already solves the fact-sparse tasks that full-context rehearsal fails (Section 4.1). **Rehearse as much as there is**: the amount of rehearsal should follow how much readable content a context holds, not a fraction fixed in advance; at an equal average amount, a content-adaptive amount beats a fixed fraction (78.0 vs. 74.6; Section 4.2). Under these principles, prior methods become special cases rather than rivals: both fix the amount before reading the context, KVzip at the whole of it and KV² at a fraction ρ , and a content-adaptive rehearsal approaches KVzip as the budget loosens.

Guided by these principles, we propose **ATTIC-KV** (ATTIC for short), named after Holmes’s brain-attic. ATTIC builds the rehearsal from three parts: **self-generated QA rehearsal**, in which the model writes the questions it expects to be asked and answers them by quoting the context; **anchors** [39], which decide *what* else to rehearse; and the length of the sentences that contain them, which decides *how much* (**content-adaptive amount**). ATTIC is training-free and changes only the rehearsal, so it plugs into any rehearsal-scoring *host method* without altering its scoring (“+ ATTIC”), and it accepts the released restore tokens of RestoreKV without retraining. Changing only the rehearsal lifts three hosts that score it

in three different ways: by attention, by gradient attribution, and with trained restore tokens. Alone, ATTIC is the best training-free method in all eight settings we test (RULER, and the 14 natural-text tasks of LongBench), and at a 3% keep ratio on RULER-4K it scores 73.4, 6.4 points above KV² and 41.9 above KVzip+, a lead that grows as the budget shrinks (Figure 1, right). Plugged into KVgrad, it raises the host by up to 17.1 points, and KVgrad + ATTIC beats every existing training-free method in all eight settings; plugged into RestoreKV+, it raises the host by up to 28.1 points and beats every existing method in all six settings where restore tokens exist (Table 1). It is also cheaper: because it rehearses a fraction of each context rather than all of it, compressing a LooGLE document and answering all of its questions takes 126.0 s against 154.8 s for KVzip+. We make three contributions:

- **Paradigm Reformulation:** We identify full-context rehearsal, the shared default of rehearsal scoring, as the cause of its failure under tight budgets; we show that *a cache keeps what it rehearses*, so that what a cache remembers depends more on what it rehearses than on how it scores the rehearsal; we observe that every existing rehearsal, the whole context or its anchors, rereads the context instead of practising what will be read out of it; and we recast query-agnostic compression as deciding what and how much to rehearse under two principles, *rehearse what will be read* and *rehearse as much as there is*, with KVzip and KV² as fixed-amount special cases.
- **Practical Solution:** We propose ATTIC, a training-free rehearsal built from self-generated QA, anchors and a content-adaptive amount, which serves alone or as a plug-in to existing hosts and composes with released restore tokens.
- **Experimental Evaluation:** On RULER with four LLMs and the natural-text tasks of LongBench with two, ATTIC is the best training-free method, every host it enters beats every existing method that trains as little as it does, and the margin widens as the budget tightens; on LooGLE, where each document is compressed once and answers all of its questions, ATTIC with restore tokens leads on both short- and long-dependency questions (Section 5).

2 Problem Setting

Compress once, answer many. Many KV caches are built before anyone knows what will be asked of them. A retrieval-augmented or search system prefills each document of its corpus once and stores the cache for every later request that retrieves it [12, 18, 34, 51]; a serving system shares the cache of a long system prompt or few-shot prefix across users; an assistant keeps the cache of a long conversation or a tool output for turns that have not happened yet. Two properties are common to all of them. **(I) One compression, many unknown queries.** A cache is compressed once and then serves every future query on its context, none of which is known when the cache is compressed. **(II) Memory grows with the number of caches.** At keep ratio r , a fixed memory holds $1/r$ times as many caches as it holds full ones: 20 times at $r = 5\%$, but only about 3 times at $r = 30\%$. Compression pays off at a few percent, and 3–5% is the regime we target.

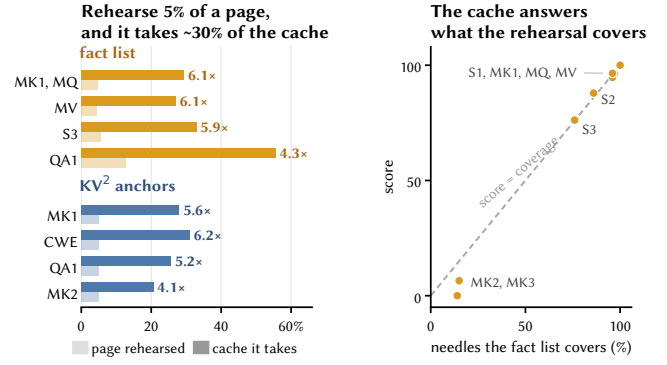


Figure 3: A cache keeps what it rehearses (Qwen3-8B, RULER-4K development split, keep ratio 5%). (Left) Share of the document that a rehearsal covers (light) and share of the kept KV entries, over all layers and heads, that fall on those positions (dark), for a semantic rehearsal (fact list) and a geometric one (KV² anchors). (Right) Rehearsing only the fact list: needle coverage of the list against the score. MK/MQ/MV: multi-key/-query/-value needles; S: single needle; CWE: common words; QA: question answering.

Query-agnostic, one-shot compression. Let $C = (c_1, \dots, c_n)$ be a document of n tokens. Prefilling C yields, at every layer $l \in [L]$ and KV head $h \in [H]$, a key-value pair $(k_i^{l,h}, v_i^{l,h})$ for each position $i \in [n]$. We call each pair an *entry* $e = (l, h, i)$ and write $\mathcal{E}$ for the set of all LHn entries. A compressor keeps a subset $\mathcal{S} \subseteq \mathcal{E}$ of size $r|\mathcal{E}|$, ranked jointly across layers and heads, and the model f then answers every query from the retained cache KV _{$\mathcal{S}$} . The goal is

$$\max_{\mathcal{S} \subseteq \mathcal{E}, |\mathcal{S}|=r|\mathcal{E}|} \mathbb{E}_{(q,a) \sim \mathcal{Q}_C} \left[U(f(q | \text{KV}_{\mathcal{S}}), a) \right], \quad (1)$$

where $\mathcal{Q}_C$ is the distribution of future questions q about C with answers a , and U is the task metric. The compression is *query-agnostic*: $\mathcal{Q}_C$ is unknown, so $\mathcal{S}$ must be computed from C alone. It is *one-shot*: a single $\mathcal{S}$ serves every draw from $\mathcal{Q}_C$.

The rehearsal-scoring paradigm. The prevailing answer to Eq. (1), introduced by KVzip [20], replaces the unknown queries with a **rehearsal**. A rehearsal text $R = (u_1, \dots, u_m)$ is prefilled after C on the full cache, and each entry is scored by the strongest attention it receives from R :

$$s_R(l, h, i) = \max_{t \in [m]} \max_{g \in \mathcal{G}(h)} \tilde{a}_{t,i}^{l,g}, \quad (2)$$

where $\mathcal{G}(h)$ is the set of query heads that share KV head h , and $\tilde{a}_{t,i}^{l,g}$ is the attention weight from rehearsal token u_t to context position i after the KVzip+ normalization [17], which rescales it by the size of the entry’s output contribution $\|W_O v_i^{l,h}\|$ relative to the hidden state of u_t . The kept set is $\mathcal{S} = \text{Top}_r(s_R)$, the $r|\mathcal{E}|$ highest-scoring entries. Several rehearsals combine entry-wise: $s_{R_1 \oplus R_2} = \max(s_{R_1}, s_{R_2})$.

Within this paradigm, a method is defined by its rehearsal, and a rehearsal by two choices: its *content* and its *amount* $|R|/n$. KVzip and its successors rehearse the whole document, $R = C$, behind a short repeat instruction and chunk by chunk for long documents [1, 17, 20, 22]; their amount is always 1. KV² [47] rehearses **anchors**,

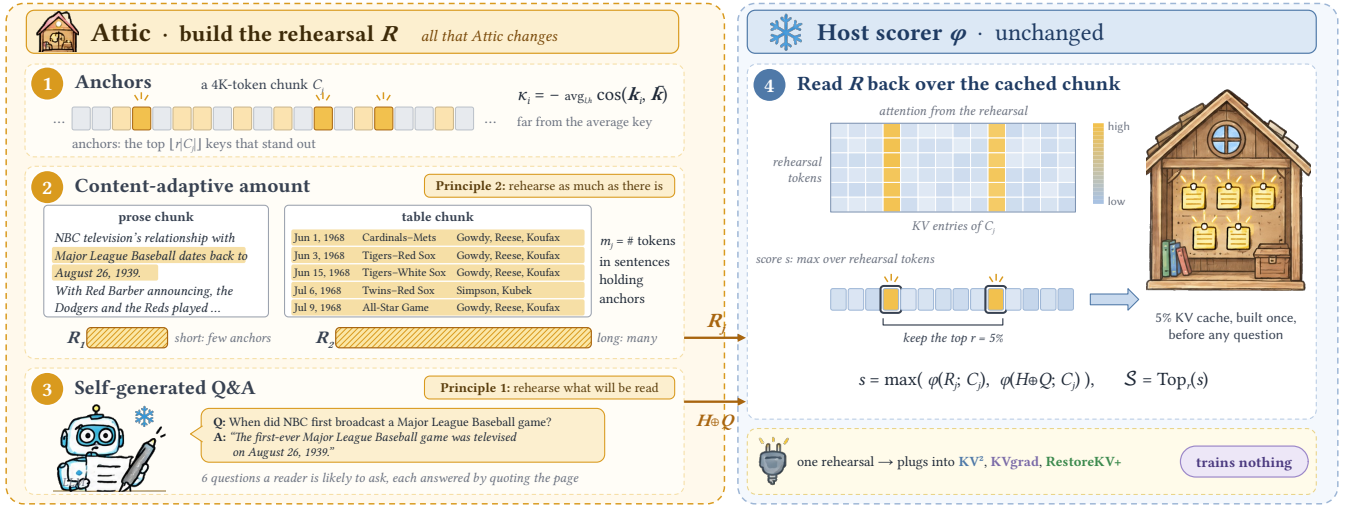


Figure 4: Overview of ATTIC. (Left) ATTIC builds the rehearsal R , the only part of compression it changes: ① the keys that stand out from the chunk’s average key are the anchors; ② the anchors are expanded to m_j tokens, the length of the sentences that hold them, so a prose chunk is rehearsed briefly and a table of facts at length (Principle 2); ③ the model writes the questions a reader is likely to ask and answers them by quoting the page (Principle 1). (Right) The host’s scorer ϕ reads the rehearsal back over the cached chunk, scores every KV entry by the attention it receives (heat map illustrative), and keeps the top r . Text is from the LooGLE page of Figure 1; the Q&A pair is the first one the model wrote for it. The same rehearsal plugs into KV^2 , $KVgrad$ and $RestoreKV+$, and nothing is trained.

$R = C[\mathcal{A}_\rho]$, where $\mathcal{A}_\rho$ holds the ρn positions with the highest KeyDiff score [39] in document order, and the fraction ρ is fixed in advance (by default, $\rho = r$). Both methods thus fix how much to rehearse before reading the document.

3 A Cache Keeps What It Rehearses

In Eq. (2), the rehearsal R stands in for every future query: whatever R contains is all the scorer knows about what will be asked. We show that R does more than inform the ranking; it decides where the budget goes. All diagnostics in this section use the development split of RULER-4K [15] with Qwen3-8B at $r = 5\%$ (13 tasks, 650 questions), disjoint from every test split in Section 5.

Evidence I: rehearsed positions are enriched in the kept budget. For a rehearsal R , let $P_R \subseteq [n]$ be the document positions that R reproduces. We compare the share of the document that R covers, $|P_R|/n$, with the share of the kept entries whose position lies in P_R . If the rehearsal merely informed the ranking, the two shares would be close; Figure 3 shows them far apart. We test two rehearsals that share nothing but the scorer. The **fact list** is semantic: with the full cache, the model copies every sentence that states a specific fact, and only this list is rehearsed (its positions are matched back to the document by 4-gram overlap). The **KV² anchors** are geometric: the tokens whose keys deviate most from the average key, chosen with no notion of a fact. For both, on fact-sparse tasks, positions covering 4.4–5.6% of the document attract 20.6–32.9% of the kept budget, an enrichment of 4.1–6.2 $\times$ and about 6 $\times$ on most tasks. A rehearsal that covers more takes more: a fact list covering 12.8% of a QA document draws 55.6% of the budget. Whichever way the rehearsal is chosen, the budget flows to what is rehearsed.

Evidence II: a cache remembers what its rehearsal covers. If the budget follows the rehearsal, a cache should answer exactly the questions that its rehearsal covers. Rehearsing only the fact list makes this testable, since we can count which needle sentences the list contains. The score follows the coverage task by task. The list covers 96–97% of the needles on MK1, MQ, and MV, and the cache scores 94.7–96.5; it covers 86% on S2 and scores 87.9, and 76% on S3 and scores 76.2. Where it misses most needles (14–15% on MK2 and MK3), it collapses to 0.0 and 6.5.

Is the loss then a matter of capacity, a 5% subset too small to hold what a question needs? An answer-aware oracle settles this. When the rehearsal is the question followed by its gold answer, attention scoring keeps a 5% subset that scores 93.5, against 94.7 without compression. The subset has room. Rehearsing the whole document instead, with the same scorer and budget, scores 37.8, and 58.9 even with the KVzip+ normalization of Eq. (2): the loss comes from what is rehearsed, not from how much is kept.

Evidence III: the answer is kept only where the rehearsal sends the budget. If the budget follows the rehearsal, it should reach the answer itself only when the rehearsal leads it there. For each of the 425 questions on the eight needle tasks (Figure 2), we measure the share of the answer’s KV entries, over all layers and heads, that the cache keeps; at $r = 5\%$, a cache that singles nothing out keeps about 5%. Whole-document rehearsal keeps 5.7–7.1% on seven of the eight tasks: the budget lands on the answer no more often than on any other token. The exception, S1, hides its needle in one sentence repeated over and over; a rereading model predicts the haystack and needs to read back only the needle, which keeps 50.1%. In a natural-text haystack nearly every token is unpredictable, so a

Table 1: Main results on the test splits at a 5% keep ratio (R-4K^{3%}: 3%). R: RULER; LB: LongBench (14 natural-text tasks). Reh.: rehearsal length, including QA pairs, as a fraction of the context (Qwen3-8B, RULER-4K test). ATTIC alone uses KV²'s scorer; a shaded row below a host adds ATTIC's rehearsal to it; $\uparrow x$: gain over KV² or the host. KV² uses the ρ selected on the development split of each model, benchmark and keep ratio; ATTIC uses one configuration throughout. [†]Restore tokens released by the RestoreKV authors (none for Qwen3-4B/14B); all other rows are training-free. Bold: above every baseline; underlined: the strongest baseline. Avg. Rank: mean rank over the columns each row has.

Group	Method	Reh.	Qwen3-8B				Llama-3.1-8B		Qwen3-4B		Qwen3-14B	
			R-4K	R-16K	R-4K ^{3%}	LB	R-4K	LB	R-4K	LB	R-4K	Avg. Rank↓
(a) Score, no rehearsal	Full cache (reference)	–	96.5	93.5	96.5	47.1	95.8	45.1	94.6	96.8	–	
	StreamingLLM [48]	0.00	11.4	12.9	6.1	28.1	15.2	29.3	10.4	20.8	10.62	
	SnapKV [26]	0.00	14.9	17.8	8.9	32.2	15.2	29.0	13.9	19.1	9.88	
	TOVA [37]	0.00	6.9	28.8	3.4	31.6	14.3	28.7	4.8	10.5	11.75	
	Expected Attention [6]	0.00	9.2	8.6	7.8	36.5	17.1	31.8	10.0	12.8	10.00	
	KeyDiff [39]	0.00	23.4	32.2	14.1	19.1	49.0	28.8	20.6	33.4	9.38	
	Compactor [5]	0.00	35.3	49.5	25.5	35.6	50.8	35.1	30.3	34.9	7.25	
(b) Rehearse everything	KVzip+ [17]	1.00	58.6	60.1	31.5	28.6	66.2	29.6	56.5	77.4	7.25	
(c) Rehearse a fixed share	KV ² [47]	0.35	76.8	74.5	<u>67.0</u>	39.1	74.8	<u>39.1</u>	<u>74.1</u>	<u>81.0</u>	4.38	
Ours	ATTIC	0.42	82.1 $\uparrow 5.3$	75.1 $\uparrow 0.6$	73.4 $\uparrow 6.4$	40.6 $\uparrow 1.5$	77.2 $\uparrow 2.4$	39.3 $\uparrow 0.2$	76.5 $\uparrow 2.4$	87.6 $\uparrow 6.6$	2.62	
(d) Gradient scoring	KVgrad [22]	1.00	72.0	<u>74.7</u>	53.5	40.3	77.1	37.9	72.1	79.5	4.62	
	+ ATTIC	0.42	83.7 $\uparrow 11.7$	80.6 $\uparrow 5.9$	70.6 $\uparrow 17.1$	41.9 $\uparrow 1.6$	80.3 $\uparrow 3.2$	39.3 $\uparrow 1.4$	81.4 $\uparrow 9.3$	88.2 $\uparrow 8.7$	1.88	
(e) Trained tokens	RestoreKV+ [1] [†]	1.00	<u>78.2</u>	73.3	47.6	33.1	85.6	37.6	–	–	5.33	
	+ ATTIC	0.42	88.9 $\uparrow 10.7$	80.6 $\uparrow 7.3$	75.7 $\uparrow 28.1$	42.4 $\uparrow 9.3$	86.4 $\uparrow 0.8$	41.6 $\uparrow 4.0$	–	–	1.00	

full reread reads them all back and spreads the budget evenly. The answer-aware oracle keeps 48.4% on average. ATTIC, the rehearsal we build in Section 4, keeps 15.0–17.0% on the five tasks that it lifts to 94–100 points, and stays at the 5% share on MK2 and MK3, the two tasks it leaves unsolved. ATTIC solves the tasks where it draws the budget to the answer, and only those. At a 3% keep ratio, whole-document rehearsal stays near chance (3.5–5.0%), while ATTIC still keeps 13.2–14.7% of the answer on the same five tasks, now 4.5 times chance: a targeted rehearsal protects the answer as the budget shrinks, so the gap widens (Sec. 5.3).

Rehearse everything, remember nothing. The three pieces of evidence identify the cause behind our headline: **a cache keeps what it rehearses**. A full-document rehearsal singles out nothing, so it spreads the budget over the whole document, and most of a document is fluent prose. Under a tight budget, that prose crowds out the few facts that will be asked. The crowding sharpens as the budget shrinks, so the gap between full and targeted rehearsal must widen as r falls; Section 5.3 confirms this prediction.

4 Two Principles and ATTIC

Section 3 reduces query-agnostic compression to the design of the rehearsal R , which has two degrees of freedom: what it contains, and how much of it there is. We state one principle for each, ground each in controlled experiments, and build ATTIC from the two.

4.1 Principle 1: Rehearse What Will Be Read

A future query reads out a few spans of a document: the value bound to a key, the date of an event, the name in a claim. The rehearsal should reproduce these spans as they would be read out, not the document. Three experiments support this principle.

(I) **Rehearsing only what is read is nearly lossless.** On the needle tasks of RULER-4K, rehearsing only the needle sentences, i.e., every fact that can be asked, under the same budget as KVzip+ scores 98.5–100 on every fact-sparse task at both $r = 5\%$ and $r =$

Table 2: Compress once, answer many on LooGLE (Qwen3-8B, keep ratio 5%): each document is compressed once, then answers all of its questions (ROUGE-L / METEOR, $\times 100$). The shaded row below RestoreKV+ adds ATTIC's rehearsal to it; $\uparrow x/\downarrow x$: change over KV² or the host. [†]Released restore tokens. Bold: above every baseline; underlined: the strongest baseline.

Method	Short dependency		Long dependency	
	ROUGE-L	METEOR	ROUGE-L	METEOR
Full cache	65.7	34.2	31.9	12.3
KVzip+	28.1	14.0	19.6	6.4
KV ²	34.3	17.2	23.6	7.8
ATTIC	36.2 $\uparrow 1.9$	18.6 $\uparrow 1.4$	22.9 $\downarrow 0.7$	7.8
RestoreKV+ [†]	<u>39.0</u>	<u>19.3</u>	<u>25.3</u>	7.2
+ ATTIC	40.9 $\uparrow 1.9$	20.7 $\uparrow 1.4$	26.1 $\uparrow 0.8$	9.2 $\uparrow 2.0$

3.125%; its lead over KVzip+ grows from 24–58 to 74–83 points (on MK1, KVzip+ falls from 44.4 to 18.5).

(II) **Reading the question is not reading the answer.** Rehearsing the question alone, without its answer, scores 47.0, against 93.5 with the answer under the same scorer. While reading a question, attention lands on the key it names; only while producing the answer does it land on the value. A rehearsal must therefore reproduce the read-out, not the request.

(III) **Self-generated read-outs carry over to natural text.** On LongBench [2], an answer-aware rehearsal keeps a 5% subset that matches the uncompressed model (44.9 vs. 44.4), so on natural text, too, the loss lies in what is rehearsed. A deployable rehearsal cannot see the questions, but the model can anticipate them: asked to write the questions a reader is likely to ask and to answer each by quoting the document, it produces read-outs of the spans it expects to be asked. Adding this **self-generated QA rehearsal** to an anchor rehearsal raises the LongBench development average from 38.2 to 39.6; Section 5.6 shows the same gain on every RULER test setting.

4.2 Principle 2: Rehearse As Much As There Is

Documents differ in how much of them can be asked. A narrative may hide one fact in a page of prose, while a table or a list of records is facts from top to bottom. The amount of rehearsal should follow this content instead of being fixed before the document is read.

(I) A content-adaptive amount beats a fixed one at equal average. On the RULER-4K development split with Qwen3-8B, we compare two anchor rehearsals that rehearse about 35% of the context on average. KV² with $\rho = 0.35$ rehearses 35% of every document and scores 74.6; the content-adaptive amount of ATTIC (Section 4.3) rehearses 11–100% per document, 34% on average, and scores 78.0. The gain comes where the content is dense: on MK2 and MK3, whose contexts are lists of key–value statements, the score rises from 27.0 to 54.0 and from 4.3 to 28.3. The amount tracks the content: over the needle-task contexts it correlates with the number of fact statements (Spearman correlation 0.59, $n = 425$), and contexts with ten or more facts are rehearsed in full (median 100%), while those with at most one are rehearsed at a median of 28%. On Llama-3.1-8B [14], it scores 76.1, against 73.9 for the best ρ .

(II) No fixed fraction is right. Tuning ρ cannot close this gap: $\rho \in \{0.25, 0.35, 0.5\}$ gives 74.0, 74.6, and 73.9, all below 78.0. The best ρ drifts with the benchmark (Section 5.5) and with the budget, as KV² itself observes [47]: on Qwen3-8B it is 0.25 at $r = 3\%$, 0.35 at 5%, and 0.9 from 10% to 30%. A fraction fixed in advance is right for some documents, benchmarks, and budgets, and wrong for the rest.

(III) KVzip and KV² are fixed-amount special cases. KVzip rehearses everything and KV² a fixed fraction, whatever the document holds; both fix the amount, and differ only in where. In ATTIC, the number of anchors equals the keep ratio, so its rehearsal grows as the budget loosens and approaches the full document: KVzip is the loose-budget limit of Principle 2, not its opposite, and at loose budgets all methods meet (Section 5.3).

4.3 ATTIC

ATTIC builds the rehearsal of Eq. (2) from the two principles: **anchors** decide what to rehearse, a **content-adaptive amount** decides how much, and a **self-generated QA rehearsal** adds the read-outs the model expects. It changes only the rehearsal; the scorer and the selection Top_r stay untouched. Figure 4 illustrates it, and Algorithm 1 (Appendix G) states it in full.

① Anchors (what). As in KV², we split C into chunks of 4,096 tokens. Each position i receives the KeyDiff score [39] $\kappa_i = -\frac{1}{H} \sum_{l,h} \cos(\mathbf{k}_i^{l,h}, \bar{\mathbf{k}}^{l,h})$, where $\bar{\mathbf{k}}^{l,h}$ is the mean normalized key of the document, so that tokens whose keys stand out from the average key score high. The anchors $\mathcal{A}_j$ of chunk C_j are its $\lceil r|C_j| \rceil$ highest-scoring positions. The anchor ratio equals the keep ratio, so ATTIC adds no ratio to tune.

② Content-adaptive amount (how much). We segment C_j into sentences, ending a sentence at any token that contains a period, question mark, exclamation mark, or line break, and let m_j be the number of tokens in the sentences that contain at least one anchor. The anchor rehearsal R_j consists of the m_j highest-scoring positions of C_j under κ , in document order: the anchors, expanded to m_j tokens. A fact-dense chunk spreads its anchors

over many sentences and receives a large m_j ; a chunk of prose concentrates them in a few and receives a small one. Sentences are a measuring stick, not the unit of rehearsal. Rehearsing the sentences themselves instead adds only 0.6 points on Qwen3-8B (78.6 vs. 78.0) and costs 4.4 on Llama-3.1-8B (71.7 vs. 76.1): the gain comes from the amount, not from the unit. The positions are rehearsed in a single pass in document order, which lets the rehearsal continue itself from local context, so that it reads back from the document mainly what it cannot predict, the needles, entities and numbers that queries ask for. Rehearsing the same positions as interleaved sparse passes drops the RULER-4K development score from 81.8 to 69.8 (Appendix C).

③ Self-generated QA rehearsal (read-outs). Before compression, the model reads the document with its full cache and generates at most 256 tokens: 6 questions a reader is likely to ask, each answered by quoting the exact words of the text (prompt in Appendix F). The output Q is rehearsed behind a one-line header H against every chunk. Because the answers quote the document, rehearsing them reproduces the read-out of the quoted spans (Section 4.1, II). Writing $\phi(R; C_j)$ for the scores that a rehearsal R gives to the entries of chunk C_j , the final score on that chunk is the entry-wise maximum $s = \max(\phi(R_j; C_j), \phi(H \oplus Q; C_j))$, and $S = \text{Top}_r(s)$.

④ Restore tokens (optional). RestoreKV [1] acts on the capacity side: 8 learned restore tokens and a LoRA adapter help the model read from a sparse cache, and the selection is left untouched. Since ATTIC acts on the selection side, the two compose directly: we attach the officially released restore weights, trained on KVzip+ selections, to the cache that ATTIC selects, and train nothing.

ATTIC as a plug-in. A rehearsal-scoring *host method* is a pair of a rehearsal and a scoring rule ϕ , where Eq. (2) is one choice of ϕ . “Host + ATTIC” keeps the host’s ϕ and replaces its rehearsal with that of ATTIC. ATTIC itself is KV² + ATTIC: the same KeyDiff anchors and scorer, with the content and amount of the rehearsal set by the two principles. KVgrad + ATTIC keeps the gradient attribution of KVgrad [22] (attention weight $\times$ output-contribution norm $\times$ a gradient norm computed on the rehearsal, originally the full document in 256-token pieces) and applies it to R_j and $H \oplus Q$ in pieces of at most 256 tokens, combined by the entry-wise maximum.

5 Experiments

We evaluate ATTIC through four research questions. **RQ1:** Does ATTIC outperform existing query-agnostic methods, both on its own and when plugged into their host methods? **RQ2:** How does its advantage change with the keep ratio? **RQ3:** Does it carry over to compress-once, answer-many caching, where a document is compressed once and then answers all of its questions, and what does the compression cost? **RQ4:** Can a fixed rehearsal ratio, however well tuned, catch up with ATTIC? We close with an ablation and a per-category view of LongBench.

5.1 Experimental Setup

Models and benchmarks. We use four open LLMs from two families: Qwen3-8B, Qwen3-4B and Qwen3-14B [49], and Llama-3.1-8B-Instruct [14]. We evaluate on three long-context benchmarks: **(1) RULER** [15] at 4K and 16K context, whose 13 subtasks cover needle retrieval, multi-hop tracing, aggregation and question answering;

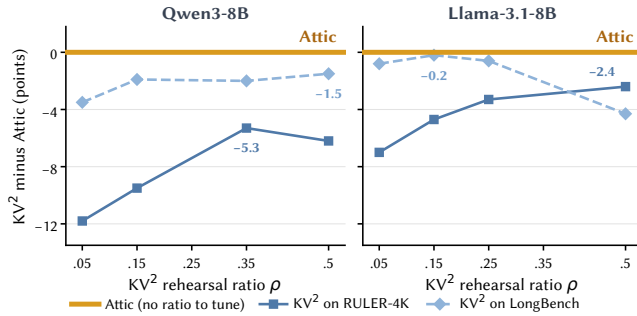


Figure 5: Tuning a fixed rehearsal ratio, even on the test split, cannot catch up with ATTIC (keep ratio 5%). Each line is KV²’s score minus ATTIC’s on the same benchmark, so ATTIC is the zero line. Every point lies below it: no single ρ gets close to ATTIC on both benchmarks at once.

(2) **LongBench** [2], whose English tasks include multi-hop QA over Wikipedia pages, QA over scientific papers, and summarization of news and reports; we report the average over its 14 natural-text tasks¹; and (3) **LooGLE** [25], long Wikipedia pages and arXiv papers, each paired with many questions, which we use for RQ3. The main keep ratio is 5%; we add 3% on Qwen3-8B RULER-4K and sweep 3–30% in RQ2. All choices are made on development splits of RULER and LongBench, and all reported numbers come from disjoint test splits (Appendix F); LooGLE is used only for testing.

Baselines. We compare against five families of query-agnostic methods, grouped as in Table 1. (a) **Scoring without rehearsal:** StreamingLLM [48], SnapKV [26], TOVA [37], Expected Attention [6], KeyDiff [39] and Compactor [5]; kvpress compresses the context before the question is appended, so each of them runs query-agnostic by construction. For rehearsal scoring we take the strongest representative of each line: (b) **rehearse everything**, KVzip+ [17]; (c) **rehearse a fixed share**, KV² [47]; (d) **gradient scoring**, KVgrad [22], which scores the same full rehearsal by gradient attribution; and (e) **trained tokens**, RestoreKV+ [1], which adds restore tokens trained by its authors to the KVzip+ selection. We run every method ourselves in kvpress [36], with the same models, splits and prompts. For KV², ρ is selected on the development split of each model, benchmark and keep ratio (Appendix D).

Configurations of ATTIC. ATTIC uses one configuration for all models, benchmarks and budgets: the anchor ratio equals the keep ratio, and the model writes 6 question–answer pairs of at most 256 tokens. These constants were fixed once on the development splits with Qwen3-8B and are never re-selected. “+ ATTIC” rows follow Section 4.3; restore tokens exist only for Qwen3-8B and Llama.

5.2 Main Results (RQ1)

Table 1 covers seven model–benchmark pairs and a 3% budget.

Obs.❶ Rehearsing nothing and rehearsing everything both collapse at tight budgets; ATTIC is the best training-free

¹We leave out LongBench’s Synthetic category, passage_count and passage_retrieval_en, which count or index paragraphs rather than ask about their content; 16-task results are in Appendix B.

method in all eight settings. Scoring without rehearsal fails first: the six heuristics score 3.4–50.8 on RULER. KeyDiff, whose score picks ATTIC’s anchors, keeps 23.4 on Qwen3-8B RULER-4K; rehearsing those anchors instead of keeping them reaches 82.1. Rehearsing everything is no cure: KVzip+ keeps 58.6 of the 96.5 uncompressed score on Qwen3-8B RULER-4K and only 31.5 at 3%, and on Qwen3-8B LongBench four of the six heuristics, which rehearse nothing, score above it (31.6–36.5 vs. 28.6). ATTIC scores 82.1 and 73.4 in the two RULER settings, 5.3 and 6.4 points above the strongest training-free baseline, KV² with a dev-tuned ρ , and on Qwen3-8B at 5% it also exceeds the trained RestoreKV+ (78.2). It leads every training-free baseline across context length, model size and natural text, by up to 6.6 points (87.6 vs. 81.0 on Qwen3-14B). Where a baseline comes close, it is a host that ATTIC improves: KVgrad is the nearest on RULER-16K, Qwen3-8B LongBench and Llama RULER-4K, and ATTIC plugged into it adds 5.9, 1.6 and 3.2 points there (Obs.❷). On Llama LongBench, KV², the host of ATTIC itself, comes within 0.2 points, but only by switching to the ratio selected for LongBench (0.15 instead of RULER’s 0.5); ATTIC gets there, as everywhere, with one configuration that transfers unchanged across models, benchmarks and budgets.

Obs.❷ Plugged into a host, ATTIC lifts it above every existing method that trains as little as it does. Plugged into KVgrad, the gradient-based host, ATTIC raises it by 11.7 points on Qwen3-8B RULER-4K, 17.1 at the 3% keep ratio, 3.2 on Llama, 9.3 on Qwen3-4B and 8.7 on Qwen3-14B; KVgrad + ATTIC beats every existing training-free method in all eight settings, and on Qwen3-4B and Qwen3-14B (81.4 and 88.2) it is the best method overall. Plugged into RestoreKV+, the trained host, it raises it in every setting, from 0.8 points on Llama RULER-4K to 28.1 at the 3% keep ratio, and RestoreKV+ + ATTIC beats every existing method in all six settings where restore tokens exist, with an average rank of 1.00. The two hosts differ in how they score a rehearsal (attention vs. gradient attribution) and in whether anything is trained, yet both improve once they rehearse what will be read, as much as there is. Full-context rehearsal fails by its target, not by its scorer or capacity.

5.3 Effect of the Budget (RQ2)

Our mechanism makes a quantitative prediction: if prose crowds facts out of a full-context rehearsal, the crowding, and with it our advantage, must grow as the budget shrinks and vanish once the budget holds everything rehearsed. Figure 1 (right) sweeps the keep ratio on Qwen3-8B RULER-4K; Table 16 extends the sweep to 30%.

Obs.❸ The advantage follows a dose–response curve: the tighter the budget, the wider the gap. The gap between ATTIC and KVzip+ is 1.1 points at a 10% keep ratio, 23.5 at 5% and 41.9 at 3%; with restore tokens, the gap to RestoreKV+ grows from 0.3 to 10.7 to 28.1 over the same ratios. At 10% both of our rows still lead their lines (94.1 vs. 93.0 for KVzip+ and 92.3 for KV²; 95.2 vs. 94.9 for RestoreKV+). At 20% and 30%, every method lies within 0.7 points of the uncompressed 96.5, the loose-budget limit of Section 4.2. The regime where compression matters most, 3–5%, is exactly where the gap is largest.

5.4 Compress Once, Answer Many (RQ3)

LooGLE reproduces this setting directly: each long document is compressed once, before any question, and then answers all of its questions, 1951 over 105 documents in the short-dependency set and 459 over 60 in the long-dependency set.

Obs.4 Compressed once per page, RestoreKV+ + ATTIC serves all of the page’s questions best, and ATTIC’s rehearsal costs less to build. RestoreKV+ + ATTIC is the best compressed method on both sets and both metrics (Table 2), above RestoreKV+ by 1.9 and 0.8 ROUGE-L and by 1.4 and 2.0 METEOR on the short- and long-dependency sets. On the short-dependency set, ATTIC alone leads all training-free methods, 1.9 ROUGE-L above KV² and 8.1 above KVzip+. Full-context rehearsal again loses most: KVzip+ keeps 28.1 of the uncompressed 65.7 ROUGE-L on short-dependency questions.

Compression cost. ATTIC is cheaper to build than the full-rehearsal methods it replaces. Measured alone on one GPU over ten LooGLE documents (about 20 questions each), compressing a page and answering all of its questions takes 126.0 s with ATTIC against 154.8 s with KVzip+, and 142.5 s with RestoreKV+ + ATTIC against 154.6 s with RestoreKV+. Writing the 6 question-answer pairs takes about 11 s, less than rehearsing a fraction of the page saves, and all of it happens once per page, off the query path.

5.5 Can a Tuned Ratio Catch Up? (RQ4)

KV² rehearses a fixed fraction ρ of each context; ATTIC lets the content decide. If, as *rehearse as much as there is* states, the right amount depends on the content, the ratio KV² needs should move with it, and no single ρ should match ATTIC everywhere.

Obs.5 The ratio KV² needs is a property of the content; ATTIC reads it off the document. Selected on each development split, KV²’s ratio for Llama is 0.5 on RULER-4K and 0.15 on LongBench (Qwen3-8B: 0.35 and 0.5). A query-agnostic cache that holds both kinds of documents would have to know in advance which kind each one is; the content-adaptive amount reads this off each document. Figure 5 scans ρ on the test splits, which gives KV² its best ratio in hindsight. On Llama, the best ratio for RULER-4K (0.5, 74.8) is the worst for LongBench (35.0), and no ratio comes within a point of ATTIC on both (77.2 and 39.3); on Qwen3-8B, the best ratios reach 76.8 and 39.1, against 82.1 and 40.6. The amount alone, without the QA rehearsal, already does what no ratio does: it beats every ρ on RULER-4K (78.8 vs. at most 76.8 on Qwen3-8B, 75.6 vs. 74.8 on Llama; Table 15), and every ρ that edges it on LongBench, by at most 0.8 points, falls 1.7–6.2 points behind it on RULER-4K.

5.6 Ablation and Task Categories

Obs.6 The content-adaptive amount replaces a tuned ratio, and self-generated QA adds most where the budget is tightest. Table 3 builds ATTIC one step at a time from two baselines. Replacing KV²’s dev-tuned ratio with the content-adaptive amount gains up to 2.0 points and loses at most 1.5, with nothing tuned; replacing the full rehearsal of RestoreKV+ with it gains up to 23.0 points under the same restore tokens. Adding self-generated QA rehearsal raises every setting, by 1.6 to 5.6 points, with the largest gain at the 3% keep ratio. The QA gain survives on top of the restore tokens

Table 3: Ablation on RULER test splits (keep ratio 5% unless marked). Each block starts from a baseline and changes one thing per row; $\uparrow_x/\downarrow_x$: change from the row above. “–”: no released restore tokens.

Rehearsal	Qwen3-8B			Llama	Qwen3-4B
	4K	16K	4K ^{3%}	4K	4K
<i>Native KV pairs</i>					
KV ² : anchors, tuned fixed ρ	76.8	74.5	67.0	74.8	74.1
adaptive amount	78.8 $\uparrow_{2.0}$	73.0 $\downarrow_{1.5}$	67.8 $\uparrow_{0.8}$	75.6 $\uparrow_{0.8}$	73.8 $\downarrow_{0.3}$
+ self-gen. QA (= ATTIC)	82.1 $\uparrow_{3.3}$	75.1 $\uparrow_{2.1}$	73.4 $\uparrow_{5.6}$	77.2 $\uparrow_{1.6}$	76.5 $\uparrow_{2.7}$
<i>With released restore tokens</i>					
RestoreKV+: full rehearsal	78.2	73.3	47.6	85.6	–
anchors, adaptive amount	85.9 $\uparrow_{7.7}$	78.8 $\uparrow_{5.5}$	70.6 $\uparrow_{23.0}$	85.3 $\downarrow_{0.3}$	–
+ self-gen. QA	88.9 $\uparrow_{3.0}$	80.6 $\uparrow_{1.8}$	75.7 $\uparrow_{5.1}$	86.4 $\uparrow_{1.1}$	–

Table 4: LongBench test results by official category (keep ratio 5%). SD/MD: single-/multi-document QA; Sum.: summarization; FS: few-shot; Avg.: the 14 natural-text tasks, as in Table 1. Shaded rows below a host add ATTIC’s rehearsal to it. $\uparrow$ Released restore tokens. Per model, bold: above every baseline; underlined: the strongest baseline.

Method	SD	MD	Sum.	FS	Code	Avg.
<i>Qwen3-8B</i> (full cache: avg. 47.1)						
KVzip+	23.8	16.3	18.5	46.3	42.9	28.6
KV ²	35.4	35.1	22.7	55.6	50.5	39.1
ATTIC	37.2	39.4	23.3	54.6	52.3	40.6
KVgrad	31.0	<u>41.7</u>	22.0	57.6	53.4	40.3
+ ATTIC	32.1	41.5	24.0	60.4	56.4	41.9
RestoreKV+ $\uparrow$	27.0	19.2	23.4	51.7	49.8	33.1
+ ATTIC	38.3	39.1	25.0	57.9	56.2	42.4
<i>Llama-3.1-8B</i> (full cache: avg. 45.1)						
KVzip+	29.0	21.0	17.2	43.1	41.5	29.6
KV ²	37.2	<u>38.1</u>	24.8	52.4	<u>45.1</u>	<u>39.1</u>
ATTIC	38.1	40.0	24.7	50.6	44.9	39.3
KVgrad	33.2	34.2	23.7	<u>56.7</u>	43.8	37.9
+ ATTIC	39.0	36.9	25.1	53.9	43.0	39.3
RestoreKV+ $\uparrow$	<u>37.2</u>	34.7	<u>25.4</u>	52.6	38.4	37.6
+ ATTIC	43.0	43.6	26.3	54.8	39.8	41.6

(1.1 to 5.1 points), which themselves add 2.3 to 9.2 to ATTIC: the two act on different sides of compression, what is selected and how much a selected subset can express.

Obs.7 ATTIC wins on the categories that questions about a document resemble: single-document QA and summarization. Table 4 breaks LongBench into its official categories. On single-document QA and summarization, which ask about the content of Wikipedia pages, papers and reports, the best result for both models is a shaded row (e.g., 43.0 vs. 37.2 for RestoreKV+ on Llama single-doc. QA). Inside KVgrad, ATTIC lifts few-shot and code by 2.8 and 3.0 on Qwen3-8B and keeps multi-document QA (41.5 vs. 41.7).

6 Related Work

Query-aware KV compression. H2O [52] keeps heavy hitters of accumulated attention, StreamingLLM [48] keeps attention sinks and a recent window, SnapKV [26] and PyramidKV [4] vote with an observation window at the end of the prompt, ChunkKV [30] keeps semantic chunks rather than single tokens, Ada-KV [10] adapts the budget across heads, and DapQ [45] places pseudo queries at

the positions of the first generated tokens. All were designed to read the question or the queries seen so far; run before it, as a compress-once cache must, they collapse (Table 1).

Query-agnostic rehearsal scoring. KVzip [20] introduced scoring by rehearsal and argues explicitly that the whole context must be repeated. Its successors keep this target and improve the scorer or its cost: the KVzap paper [17] introduces KVzip+, which normalizes by the value contribution, and KVzap, Fast KVzip [19] and EchoPress [29] approximate the full-rehearsal score cheaply, and KVgrad [22] replaces attention with a gradient attribution. All are instances of the **rehearse-everything default**, and their scorers compose with our rehearsal. KV² [47] rehearses KeyDiff anchors [39] at a fixed ratio ρ ; it is the **fixed-ratio special case of rehearsal as much as there is**, and like full-context rehearsal it rehearses a copy of the context, and its own observation that the best ρ drifts with the budget is what the principle predicts. InfiniPot [21] scores the cache with a short fixed “catalyst” prompt, and TaskPress [40] replaces the rehearsal with a two-to-three-sentence task description written by an LLM; that the TaskPress probe already beats full-context rehearsal at 50–75% eviction is consistent with *rehearse what will be read*, though both probes are fixed in advance rather than drawn from the document.

Capacity-side and learned caches. RestoreKV [1] enlarges what a kept subset can express with restore tokens trained on KVzip+ selections, rather than choosing the subset. Cartridges [9] distill a document into a trained cache via self-study, Cartridges++ [23] guards it against off-context derailment, Attention Matching [53] fits compact keys and values to reference queries, ARC-KV [43] amortizes the anchor search of such compaction, and PatchKV [24] compensates eviction in weight space. These methods share our compress-once, query-many setting; Attention Matching and ARC-KV draw most of their reference queries from a verbatim repeat of the context, another instance of the rehearse-everything default; Attention Matching adds a few self-study passes in which the model asks and answers its own questions, the closest relative of our question–answer rehearsal. ATTIC keeps native KV and trains nothing.

Study strategies for language models. Retrieval practice, answering one’s own questions, is among the most effective ways for people to learn, and rereading among the least [8, 42]. Language models that must learn documents into their weights benefit from the same contrast: Cartridges [9] find that training on the raw corpus memorizes the text but does not generalize, while self-study conversations do, and Active Reading [28] lets the model generate its own study strategies, including active recall. Predicting the questions a document will be asked also has a precedent in retrieval, where doc2query [35] expands documents with them. ATTIC brings the contrast to a setting where nothing is trained: the rehearsal selects which KV entries stay, and the model quizzes itself instead of rereading.

KV cache reuse for retrieval and serving. Prompt Cache [12], CacheBlend [51], RAGCache [18] and TurboRAG [34] precompute document KV caches and reuse them across requests, LMCache [31] and Mooncake [41] store them across GPU, CPU and disk tiers, solving fusion and latency for full-size caches whose storage grows

with the corpus; SCBench [27] shows that compression conditioned on one request degrades when the same cache serves later ones, which is why such caches must be compressed query-agnostically; CacheGen [32] encodes stored caches into compact bitstreams, and since encoding shrinks each kept entry and eviction how many are kept, the two multiply; ATTIC is the eviction that lets them scale to large corpora.

Other importance signals. Expected Attention [6] and CapKV [50] model future queries statistically, as a Gaussian over hidden states or an information-capacity objective. Behavior-Preserving KV compression [16] also questions the reconstruction target of KVzip, replacing it with a next-token KL score at available query positions, LORE-KV [3], Lookahead Q-Cache [46] and SpecKV [11] draft the response to a known request and score entries by the attention the draft pays them; that response-side queries beat prompt-side ones is the query-aware counterpart of *rehearse what will be read*. Judge Q [33] trains soft query tokens whose attention imitates that of generated responses, and GVote [44] samples synthetic queries from hidden-state statistics. Compactor [5] scores entries without a rehearsal, by key leverage and non-causal attention, and calibrates the keep ratio to each context. Back from the Future [13] evicts entries that later tokens predict well, by a counter-causal pass over the cache. LLMingua-2 [38] compresses the text itself before prefill. ATTIC keeps rehearsal scoring and changes what, and how much, is rehearsed.

7 Conclusion

A KV cache that is compressed before its questions are known must decide, from the context alone, what to remember. The rehearsal-scoring paradigm decides by rereading: it rehearses everything and remembers nothing, because a cache keeps what it rehearses, and a whole-context rehearsal fills the budget with prose that no query reads. Two principles turn this failure into a design rule, *rehearse what will be read* and *rehearse as much as there is*, and ATTIC realizes them without training, by letting the model quiz itself on the context. Alone, ATTIC is the best training-free method; inside KVgrad and RestoreKV+, it lifts each above every existing method that trains as little; and its lead grows as the budget shrinks. Since what a cache remembers depends more on what it rehearses than on how it scores, the same principles suggest how to choose the proxy queries of any compressor that stands in for unknown questions, from token eviction to learned compaction. Holmes’s attic has no elastic walls, and neither does a compressed cache. The skilful workman does not carry in everything he comes across; he carries in the tools the work will need, and as many as the work needs.

References

- [1] Changwoo Baek, Seungjun Shin, and Kyeongbo Kong. 2026. RestoreKV: Recovering Full-Cache Behavior Under Aggressive Query-Agnostic KV Cache Eviction. *arXiv preprint arXiv:2608.01247* (2026).
- [2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2308.14508.
- [3] Ahsan Bilal, Muhammad Ahmed Mohsin, Muhammad Umer, Wajih Hassan Raza, Atta Ul Asad, Young D. Kwon, Michal Valko, and Dean F. Hougen. 2026. Monte Carlo Estimation for KV Cache Eviction. *arXiv preprint arXiv:2610.07643* (2026).

- [4] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069* (2024).
- [5] Vivek Chari and Benjamin Van Durme. 2025. Compactor: Calibrated Query-Agnostic KV Cache Compression with Approximate Leverage Scores. *arXiv preprint arXiv:2507.08143* (2025).
- [6] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. 2025. Expected Attention: KV Cache Compression by Estimating Attention from Future Queries Distribution. *arXiv preprint arXiv:2510.00636* (2025).
- [7] Arthur Conan Doyle. 1887. *A Study in Scarlet*. Ward Lock & Co, London.
- [8] John Dunlosky, Katherine A. Rawson, Elizabeth J. Marsh, Mitchell J. Nathan, and Daniel T. Willingham. 2013. Improving Students' Learning With Effective Learning Techniques: Promising Directions From Cognitive and Educational Psychology. *Psychological Science in the Public Interest* 14, 1 (2013), 4–58.
- [9] Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Ré. 2025. Cartridges: Lightweight and General-Purpose Long Context Representations via Self-Study. *arXiv preprint arXiv:2506.06266* (2025).
- [10] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [11] Kevin Galim, Ethan Ewer, Wonjun Kang, Minjae Lee, Hyung Il Koo, and Kangwook Lee. 2025. Draft-based Approximate Inference for LLMs. *arXiv preprint arXiv:2506.08373* (2025).
- [12] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems (MLSys)*. arXiv:2311.04934.
- [13] Stephen Gould and Anton van den Hengel. 2026. Back from the Future: Key-Value Cache Management by Counter-Causal Surprise. *arXiv preprint arXiv:2607.27600* (2026).
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [15] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesht, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models?. In *Conference on Language Modeling (COLM)*. arXiv:2404.06654.
- [16] Doo Hwan Hwang, Junyoung Jang, Junho Na, Hosung Lim, and Kee-Eung Kim. 2026. Behavior-Preserving KV Cache Compression. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2610.06479.
- [17] Simon Jégou and Maximilian Jeblick. 2026. KVzap: Fast, Adaptive, and Faithful KV Cache Pruning. *arXiv preprint arXiv:2601.07891* (2026).
- [18] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.12457* (2024).
- [19] Jang-Hyun Kim, Dongyoon Han, and Sangdoo Yun. 2026. Fast KVzip: Efficient and Accurate LLM Inference with Gated KV Eviction. *arXiv preprint arXiv:2601.17668* (2026).
- [20] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. 2025. KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2505.23416.
- [21] Minsoo Kim, Kyuhong Shim, Jungwook Choi, and Simyung Chang. 2024. InfiniPot: Infinite Context Processing on Memory-Constrained LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [22] Jihwan Kwak, Sunghwan Joo, Jung Yoon Hwang, Jaeseok Byun, and Taesup Moon. 2026. KVgrad: Query-Agnostic KV Cache Eviction via Gradient-based Global Importance Scoring. In *ICML 2026 Workshop on Resource-Adaptive Foundation Model Inference (AdaptFM)*. <https://openreview.net/forum?id=cg1wTCJjkk>
- [23] Sonia Laguna, Joao Monteiro, Marco Cuturi, Pierre Ablin, and Eleonora Gualdoni. 2026. Cartridges++: KV Cache Compression without Off-Context Derailment. *arXiv preprint arXiv:2609.35621* (2026).
- [24] Chanryeol Lee, Chanhyuk Lee, Yeonwoo Choi, Donggyun Kim, and Seunghoon Hong. 2026. PatchKV: Weight-Space Compensation of KV Cache. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2609.39329.
- [25] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2024. LooGLE: Can Long-Context Language Models Understand Long Contexts?. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2311.04939.
- [26] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2404.14469.
- [27] Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. 2025. SCBench: A KV Cache-Centric Analysis of Long-Context Methods. In *International Conference on Learning Representations (ICLR)*.
- [28] Jessy Lin, Vincent-Pierre Berges, Xilun Chen, Wen-tau Yih, Gargi Ghosh, and Barlas Oğuz. 2025. Learning Facts at Scale with Active Reading. *arXiv preprint arXiv:2508.09494* (2025).
- [29] Jiawei Lin, Saibo Geng, and Thomas Bourgeat. 2026. EchoPress: Query-Agnostic KV Cache Pruning via Virtual Context Reconstruction. *arXiv preprint arXiv:2610.00412* (2026).
- [30] Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. 2025. ChunkKV: Semantic-Preserving KV Cache Compression for Efficient Long-Context LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2502.00299.
- [31] Yuhua Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).
- [32] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*.
- [33] Yijun Liu, Yixuan Wang, Yuzhuang Xu, Shiyu Ji, Yang Xu, Qingfu Zhu, and Wanxiang Che. 2026. Judge Q: Trainable Queries for Optimized Information Retention in KV Cache Eviction. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [34] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2024. TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text. *arXiv preprint arXiv:2410.07590* (2024).
- [35] Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. 2019. Document Expansion by Query Prediction. *arXiv preprint arXiv:1904.08375* (2019).
- [36] NVIDIA. 2024. KVPress: KV Cache Compression Library. <https://github.com/NVIDIA/kvpress>. GitHub repository, accessed October 2026.
- [37] Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. 2024. Transformers are Multi-State RNNs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.
- [38] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- [39] Junyoung Park, Dalton Jones, Matthew J. Morse, Raghavv Goel, Mingyu Lee, and Chris Lott. 2025. KeyDiff: Key Similarity-Based KV Cache Eviction for Long-Context LLM Inference in Resource-Constrained Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2504.15364.
- [40] Wonpyo Park and Seung-won Hwang. 2026. TaskPress: Query-Agnostic KV Cache Compression via Task-Guided Pruning. *arXiv preprint arXiv:2608.03276* (2026).
- [41] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. In *23rd USENIX Conference on File and Storage Technologies (FAST)*. arXiv:2407.00079.
- [42] Henry L. Roediger and Jeffrey D. Karpicke. 2006. Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention. *Psychological Science* 17, 3 (2006), 249–255.
- [43] Zheyu Shen, Guanhua Wang, Dezhao Tu, Mengchi Zhang, Yanjia Li, Adnan Aziz, Chunqiang Tang, and Ang Li. 2026. ARC-KV: Amortizing Anchor Search for Reconstruction-Based KV Cache Compaction. *arXiv preprint arXiv:2609.36835* (2026).
- [44] Chenxia Tang, Jianchun Liu, Hongli Xu, and Liusheng Huang. 2025. Adaptive KV-Cache Compression without Manually Setting Budget. *arXiv preprint arXiv:2509.03136* (2025).
- [45] Zhenxu Tian, Yi Su, Juntao Li, and Min Zhang. 2026. Where Matters More Than What: Decoding-aligned KV Cache Compression via Position-aware Pseudo Queries. *arXiv preprint arXiv:2603.11564* (2026).
- [46] Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. 2025. Lookahead Q-Cache: Achieving More Consistent KV Cache Eviction via Pseudo Query. *arXiv preprint arXiv:2505.20334* (2025).
- [47] Johannes Wesch, Danni Liu, and Jan Niehues. 2026. KV²: A Self-Refining KV Cache. *arXiv preprint arXiv:2610.03198* (2026).
- [48] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.17453.
- [49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

- [50] Jiaming Yang, Chenwei Tang, Liangli Zhen, and Jiancheng Lv. 2026. Rethinking KV Cache Eviction via a Unified Information-Theoretic Objective. In *International Conference on Machine Learning (ICML)*. arXiv:2604.25975.
- [51] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*. arXiv:2405.16444.
- [52] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.14048.
- [53] Adam Zweiger, Xinghong Fu, Han Guo, and Yoon Kim. 2026. Fast KV Compaction via Attention Matching. *arXiv preprint arXiv:2602.16284* (2026).

A Per-Subtask RULER Results

Tables 5–10 break each RULER setting of Table 1 down into its 13 subtasks; the Avg. column is the number reported there. KV² at its default ratio ($\rho=0.05$) appears in Table 15.

B Per-Task LongBench Results

The main text reports the average over the 14 natural-text tasks of LongBench. Table 11 and Table 12 give the per-task scores together with that average (Avg₁₄) and the average over all 16 English tasks (Avg₁₆), which adds the two synthetic tasks (passage_count and passage_retrieval_en); Appendix C shows why coherent rehearsal reads the content of a passage rather than its label.

C What Coherent Rehearsal Reads

A cache keeps what it rehearses; this appendix asks what, *within* a rehearsal, the model actually reads, and why ATTIC rehearses its positions as one coherent stream (Section 4.3).

Coherent rehearsal reads only what it cannot predict. A rehearsal score is the attention that rehearsed tokens pay back to the context. When the rehearsal is coherent text, as ATTIC’s anchor rehearsal and self-generated QA pairs are, the model continues the stream from its own local context and looks back into the document only for tokens it cannot guess: needle values, entities, numbers. These are exactly the tokens that questions about a document ask for, and they are kept. Structure that is **predictable but must be read** meets the opposite fate: sequentially numbered passage labels (“Paragraph 7”) and the labels of in-context examples are produced from the stream itself, draw no attention back to the document, and are lost.

Table 13(a) shows this on LongBench passage retrieval, whose answer is a “Paragraph *N*” label. Because every sentence of a Wikipedia passage is an askable fact, the content-adaptive amount rehearses about half of the context on this task. As rehearsal grows from isolated anchors (KV², 5%) into coherent text, the kept fraction of label tokens falls from about 5× the 5% base rate to the base rate itself, and the score falls with it (Qwen3-8B: 77 → 40; Llama-3.1-8B: 100 → 30).

Coherence, not amount, decides which tokens survive. To isolate coherence, we keep ATTIC’s rehearsed positions fixed and only break the stream: the positions of each chunk are split by a stride into interleaved sparse passes, each rehearsed separately, with the per-token score taken as the maximum over passes. The rehearsed token count is unchanged, but no pass can be continued

from local context, so every rehearsed token has to be read back. Table 13(b) shows both sides of the trade. Labels return: passage retrieval on Qwen3-8B rises from 59 to 78.7, level with KV² at its default ratio on the same split (79). Needles leave: RULER drops from 81.8 to 69.8, with multi-key-2 falling from 57 to 5 and multi-value from 98 to 59. Under a tight budget the two kinds of tokens compete for the same slots: coherent rehearsal spends the budget on what the model cannot predict, while sparse rehearsal spreads it over everything rehearsed and crowds the needles out.

A cache keeps what its rehearsal reads. The finding refines our mechanism from “a cache keeps what it rehearses” to *a cache keeps what its rehearsal reads*. Questions about a document target facts, entities and numbers, which is what coherent rehearsal reads; it is the reading mode ATTIC uses. The passage labels of LongBench’s two synthetic tasks (Tables 11–12) are predictable structure, which coherent rehearsal does not read.

Table 13: What coherent rehearsal reads (development split, keep ratio 5%). (a) Passage retrieval, first 10 questions: as rehearsal grows into coherent text, the kept fraction of “Paragraph *N*” label tokens falls to the 5% base rate and the score follows. (b) Qwen3-8B: breaking the same rehearsal into sparse passes recovers labels but loses RULER needles. Amt.: rehearsal amount (fraction of context); Label: kept fraction of label tokens; content-adaptive: ATTIC’s amount without QA rehearsal. PR: passage retrieval; MK2/MK3: multi-key 2/3; MV: multi-value.

(a) Rehearsal	Qwen3-8B			Llama-3.1-8B		
	Amt.	Label	Score	Amt.	Label	Score
KV ² (default)	5%	~25%	77	5%	~20%	100
KV ² ($\rho=0.35$)	35%	~7%	50	35%	~4.5%	70
Content-adaptive	53%	~5%	40	40%	~4%	30

(b) Qwen3-8B	PR	RULER	MK2	MK3	MV
Coherent (ATTIC)	59	81.8	57	30	98
Sparse multi-pass	78.7	69.8	5	7	59

D Selecting ρ for KV²

KV² rehearses a fixed fraction ρ of anchor tokens per chunk. Its authors set ρ to the keep ratio by default; since this default is far from optimal on our development splits, we give KV² a tuned ratio in the main table. For each model we evaluate a grid of ρ on the RULER-4K development split and use the best value on RULER; on LongBench, ρ is selected in the same way on the LongBench development split, and LooGLE, which has no development split, uses the RULER choice; at the 3% keep ratio and in the budget sweep, ρ is re-selected for each keep ratio (Table 14). The selected ratios are 0.35 for Qwen3-8B, 0.5 for Llama-3.1-8B and 0.5 for Qwen3-4B at a 5% keep ratio, 0.25 for Qwen3-8B at 3%, and 0.9 at 10%, 20% and 30%; on LongBench, 0.15 for Llama-3.1-8B and 0.5 for Qwen3-8B. For Qwen3-14B, ρ is chosen from {0.25, 0.35, 0.5} in the same way, and $\rho=0.5$ is selected. ATTIC uses no such search: its anchor ratio is the keep ratio in every setting. Table 15 lists the test-split ρ scan plotted in Figure 5; it is used only for that analysis and not for selecting ρ .

Table 5: Per-subtask results, Qwen3-8B, RULER-4K (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	99.2	96.8	84.0	75.0	96.5
KVzip+	100.0	63.8	61.4	42.0	60.5	9.4	42.1	56.3	100.0	58.1	68.8	48.0	51.9	58.6
KV ²	100.0	95.7	100.0	94.0	34.9	1.9	95.8	96.6	100.0	85.2	91.4	58.0	44.2	76.8
ATTIC	100.0	100.0	100.0	98.0	51.2	32.1	97.1	98.9	100.0	92.9	93.0	58.0	46.2	82.1
KVgrad	100.0	85.1	95.5	74.0	41.9	20.8	90.0	85.2	99.3	81.3	74.7	34.0	53.9	72.0
+ ATTIC	100.0	100.0	100.0	100.0	74.4	54.7	98.3	98.9	100.0	79.4	89.8	46.0	46.2	83.7
RestoreKV+ [†]	100.0	74.5	77.3	68.0	93.0	83.0	72.5	92.6	100.0	62.5	74.7	64.0	53.9	78.2
+ ATTIC	100.0	100.0	100.0	100.0	72.1	81.1	99.2	98.9	100.0	95.8	94.1	62.0	51.9	88.9

Table 6: Per-subtask results, Qwen3-8B, RULER-16K (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	98.0	100.0	100.0	100.0	100.0	100.0	84.2	92.5	72.0	69.2	93.5
KVzip+	100.0	80.9	88.6	46.0	23.3	7.6	57.9	69.9	100.0	20.6	89.3	42.0	55.8	60.1
KV ²	100.0	100.0	100.0	90.0	34.9	9.4	94.6	98.9	100.0	50.0	93.0	42.0	55.8	74.5
ATTIC	100.0	100.0	100.0	96.0	27.9	7.6	96.3	99.4	100.0	67.9	91.4	38.0	51.9	75.1
KVgrad	100.0	91.5	97.7	96.0	51.2	20.8	87.5	93.8	100.0	55.2	88.2	38.0	51.9	74.7
+ ATTIC	100.0	100.0	100.0	100.0	65.1	34.0	95.4	100.0	100.0	77.3	90.3	32.0	53.9	80.6
RestoreKV+ [†]	100.0	95.7	95.5	56.0	55.8	50.9	74.2	91.5	100.0	21.9	90.3	56.0	65.4	73.3
+ ATTIC	100.0	100.0	100.0	98.0	53.5	47.2	97.5	99.4	100.0	66.3	93.6	40.0	51.9	80.6

Table 7: Per-subtask results, Qwen3-8B, RULER-4K, keep ratio 3% (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	99.2	96.8	84.0	75.0	96.5
KVzip+	100.0	31.9	25.0	6.0	7.0	0.0	23.8	11.4	100.0	6.3	19.4	38.0	40.4	31.5
KV ²	100.0	83.0	100.0	82.0	14.0	0.0	90.0	88.1	100.0	59.4	82.3	38.0	34.6	67.0
ATTIC	100.0	100.0	100.0	94.0	14.0	1.9	92.9	96.0	100.0	86.7	87.6	44.0	36.5	73.4
KVgrad	100.0	48.9	54.6	64.0	18.6	3.8	66.7	59.1	97.0	57.5	53.2	28.0	44.2	53.5
+ ATTIC	100.0	100.0	100.0	94.0	18.6	0.0	94.6	97.2	99.6	54.2	85.5	38.0	36.5	70.6
RestoreKV+ [†]	100.0	53.2	25.0	32.0	41.9	11.3	55.4	46.0	100.0	20.4	53.8	36.0	44.2	47.6
+ ATTIC	100.0	100.0	100.0	98.0	18.6	17.0	96.3	98.3	100.0	85.8	89.8	42.0	38.5	75.7

Table 8: Per-subtask results, Llama-3.1-8B, RULER-4K (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	99.3	99.6	96.8	84.0	65.4	95.8
KVzip+	100.0	93.6	72.7	76.0	93.0	11.3	57.5	83.5	97.0	8.1	76.3	40.0	51.9	66.2
KV ²	100.0	97.9	100.0	98.0	76.7	5.7	80.0	93.2	98.1	29.8	89.3	54.0	50.0	74.8
ATTIC	100.0	100.0	97.7	98.0	51.2	9.4	90.4	98.9	98.5	68.1	94.6	50.0	46.2	77.2
KVgrad	100.0	95.7	93.2	98.0	97.7	54.7	85.8	94.9	98.9	29.4	87.1	24.0	42.3	77.1
+ ATTIC	100.0	100.0	100.0	100.0	60.5	56.6	99.2	100.0	98.9	44.2	88.2	48.0	48.1	80.3
RestoreKV+ [†]	100.0	100.0	95.5	94.0	100.0	92.5	89.2	97.7	99.6	18.5	84.4	78.0	63.5	85.6
+ ATTIC	100.0	100.0	97.7	100.0	65.1	81.1	97.5	100.0	99.6	72.7	93.6	58.0	57.7	86.4

Table 14: KV² development results for each candidate ρ (RULER-4K development split; LB: LongBench development split, 14 natural-text tasks); the selected value is in bold.

Setting	ρ : development score
Qwen3-8B, 5%	0.05: 68.0; 0.25: 74.0; 0.35: 74.6 ; 0.5: 73.9
Llama-3.1-8B, 5%	0.05: 68.1; 0.1: 70.6; 0.25: 73.7; 0.35: 73.0; 0.5: 73.9
Llama-3.1-8B, LB	0.05: 37.2; 0.15: 39.7 ; 0.5: 37.1
Qwen3-8B, LB	0.05: 37.9; 0.15: 38.5; 0.35: 38.0; 0.5: 38.6
Qwen3-4B, 5%	0.25: 70.85; 0.35: 70.34; 0.5: 70.87
Qwen3-14B, 5%	0.25: 76.5; 0.35: 79.4; 0.5: 80.1
Qwen3-8B, 3%	0.15: 64.71; 0.25: 65.33 ; 0.35: 62.41
Qwen3-8B, 10%	0.1: 75.5; 0.35: 83.6; 0.5: 86.0; 0.7: 88.7; 0.9: 91.0
Qwen3-8B, 20%	0.2: 86.9; 0.5: 91.4; 0.7: 92.9; 0.9: 93.2
Qwen3-8B, 30%	0.5: 94.0; 0.7: 94.5; 0.9: 94.6

Table 15: KV² ρ scan on the test splits (keep ratio 5%; Long-Bench: 14 natural-text tasks), the data of Figure 5. Amount: ATTIC’s content-adaptive amount without the QA rehearsal. “–”: not run.

Setting	KV ² , $\rho =$						Amount	ATTIC
	0.05	0.15	0.25	0.35	0.5			
Qwen3-8B R-4K	70.3	72.6	–	76.8	75.9	78.8		82.1
Qwen3-8B LB	37.1	38.7	–	38.6	39.1	38.6		40.6
Llama R-4K	70.2	72.5	73.9	–	74.8	75.6		77.2
Llama LB	38.5	39.1	38.7	–	35.0	38.3		39.3

E Budget Sweep

Table 16 gives the numbers behind Figure 1 (Qwen3-8B, RULER-4K test split). The 3% and 5% columns are those of Table 1; KV² uses the ratio selected on the development split at each keep ratio (Table 14).

Table 9: Per-subtask results, Qwen3-4B, RULER-4K (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	94.0	88.2	76.0	71.2	94.6
KVzip+	100.0	72.3	65.9	42.0	72.1	5.7	32.1	51.1	100.0	36.7	68.8	46.0	42.3	56.5
KV ²	100.0	97.9	100.0	94.0	58.1	13.2	84.2	78.4	100.0	56.0	78.0	50.0	53.9	74.1
ATTIC	100.0	100.0	100.0	100.0	55.8	7.6	97.1	84.1	100.0	74.4	81.2	48.0	46.2	76.5
KVgrad	100.0	87.2	100.0	78.0	55.8	34.0	72.9	71.6	99.6	75.4	72.6	32.0	57.7	72.1
+ ATTIC	100.0	100.0	100.0	100.0	83.7	45.3	95.0	94.3	100.0	72.7	79.6	42.0	46.2	81.4

Table 10: Per-subtask results, Qwen3-14B, RULER-4K (test split).

Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg.
Full cache	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	99.6	96.2	92.0	71.2	96.8
KVzip+	100.0	87.2	88.6	86.0	95.4	34.0	64.2	86.4	99.6	60.6	81.2	66.0	57.7	77.4
KV ²	100.0	91.5	100.0	96.0	76.7	9.4	95.8	99.4	100.0	84.2	91.9	66.0	42.3	81.0
ATTIC	100.0	100.0	100.0	100.0	79.1	62.3	100.0	100.0	87.9	91.9	68.0	50.0	87.6	
KVgrad	100.0	95.7	97.7	88.0	74.4	45.3	92.1	77.8	100.0	90.4	80.1	34.0	57.7	79.5
+ ATTIC	100.0	100.0	100.0	98.0	83.7	77.4	100.0	100.0	100.0	91.7	90.3	48.0	57.7	88.2

Table 11: Per-task LongBench results, Qwen3-8B (test split, keep ratio 5%). Avg₁₄: the 14 natural-text tasks used in Table 1; Avg₁₆: all 16 tasks.

Method	NQA	Qsp	MFQA	HQA	2WQA	Musq	GovR	QMS	MNews	TREC	TQA	SSum	LCC	RB-P	PCnt	PRet	Avg ₁₄	Avg ₁₆
Full cache	31.9	45.6	54.7	68.1	56.2	34.6	32.6	24.5	26.4	38.7	85.5	39.9	63.9	57.8	9.3	94.6	47.1	47.7
KVzip+	21.3	17.2	33.0	25.8	13.7	9.4	22.3	20.5	12.7	24.0	79.9	35.1	23.4	62.4	2.7	32.0	28.6	27.2
KV ²	33.0	32.8	40.5	52.9	35.8	16.7	26.7	21.2	20.3	45.3	85.2	36.2	37.5	63.6	4.0	50.0	39.1	37.6
ATTIC	27.7	35.5	48.5	56.4	41.9	19.7	28.7	21.8	19.6	42.7	85.8	35.4	44.1	60.4	2.7	45.8	40.6	38.5
KVgrad	24.9	29.5	38.6	55.8	45.8	23.5	25.5	21.2	19.3	50.7	85.3	36.8	48.3	58.6	9.3	94.1	40.3	41.7
+ ATTIC	24.1	26.9	45.4	57.2	43.1	24.2	27.9	22.2	22.0	59.3	84.5	37.3	56.0	56.7	2.7	79.3	41.9	41.8
RestoreKV+ [†]	19.2	26.9	34.8	27.3	20.1	10.4	26.0	22.6	21.5	40.0	78.7	36.5	38.5	61.2	2.7	39.1	33.1	31.6
+ ATTIC	30.0	35.5	49.5	53.3	41.8	22.2	28.5	23.6	23.0	50.7	86.7	36.2	53.0	59.5	4.0	44.7	42.4	40.1

Table 12: Per-task LongBench results, Llama-3.1-8B (test split, keep ratio 5%). Avg₁₄: the 14 natural-text tasks used in Table 1; Avg₁₆: all 16 tasks.

Method	NQA	Qsp	MFQA	HQA	2WQA	Musq	GovR	QMS	MNews	TREC	TQA	SSum	LCC	RB-P	PCnt	PRet	Avg ₁₄	Avg ₁₆
Full cache	40.5	49.4	57.6	56.7	59.0	32.3	33.4	24.8	27.6	30.0	88.3	40.9	49.5	41.7	10.0	100.0	45.1	46.4
KVzip+	33.8	19.4	33.8	25.2	26.5	11.3	17.9	20.3	13.5	10.0	86.2	33.2	27.5	55.5	0.0	34.0	29.6	28.0
KV ²	31.5	36.0	44.2	46.6	43.7	24.1	28.4	22.5	23.6	31.0	86.3	39.9	35.4	54.8	6.0	88.0	39.1	40.1
ATTIC	33.3	35.3	45.8	45.5	49.2	25.4	27.7	22.4	23.9	26.0	86.9	39.0	36.0	53.7	2.0	50.0	39.3	37.6
KVgrad	32.0	32.4	35.1	39.4	38.0	25.3	25.9	22.5	22.8	44.0	85.3	40.9	40.8	46.8	8.0	98.0	37.9	39.8
+ ATTIC	29.6	39.3	48.1	47.7	42.4	20.5	27.4	23.1	24.7	38.0	82.9	40.7	40.2	45.7	10.0	82.0	39.3	40.1
RestoreKV+ [†]	30.7	31.2	49.8	38.9	46.6	18.5	27.5	22.9	25.8	52.0	89.2	16.5	29.0	47.8	0.0	46.0	37.6	35.8
+ ATTIC	33.9	43.2	51.9	49.8	54.6	26.5	28.7	23.3	26.8	42.0	85.9	36.5	29.1	50.4	4.0	58.0	41.6	40.3

Table 16: RULER-4K test score of Qwen3-8B across keep ratios. [†]Uses released restore tokens. Shaded rows use ATTIC’s rehearsal. Bold: above every baseline; underlined: the strongest baseline.

Method	3%	5%	10%	20%	30%
Full cache	96.5	96.5	96.5	96.5	96.5
KVzip+	31.5	58.6	93.0	96.0	96.2
KV ² (dev-tuned ρ)	<u>67.0</u>	76.8	92.3	95.9	95.8
RestoreKV+ [†]	47.6	<u>78.2</u>	<u>94.9</u>	<u>96.5</u>	<u>96.4</u>
ATTIC	73.4	82.1	94.1	96.1	96.0
RestoreKV+ + ATTIC [†]	75.7	88.9	95.2	96.4	96.2

F Implementation Details

Framework. All methods run in kvpress [36] on the same models, prompts and splits. Context is truncated to 32K tokens on LongBench and LooGLE. KVgrad, KVzip+ and RestoreKV+ use their kvpress implementations; KV² is implemented in the same scoring

and eviction code as KVzip+. Fixed-block retention in the style of ChunkKV [30], applied to KVzip+ scores, reaches only 22.5 on Qwen3-8B RULER-4K, so we leave it out of the tables.

Development and test splits. On RULER, the development split is a random 10% of each context length (seed 42), and the test split draws the same number of questions from the remainder (seed 7). On LongBench, each task is shuffled (seed 42); the first 25 questions form the development split and the next 75 (Qwen) or 50 (Llama) form the test split.

Rehearsal scoring and anchors. Every rehearsal-based method shares the KVzip+ scoring and eviction of Eq. (2), combining the scores of several rehearsed texts by an element-wise maximum. Anchors and the content-adaptive amount follow Section 4.3, with chunks of 4096 tokens rehearsed and scored separately.

Self-generated QA rehearsal. The QA pairs are decoded greedily from the prompt below, appended to the context after two new-lines and the same for all models and benchmarks:

Write 6 questions a reader is likely to ask
about the text above. After each question,
answer it by quoting the exact words from the
text that contain the answer. Format each as:
Q: ...
A: ...

The header H is “Questions a reader may ask about the previous context, with answers quoted from it”.

Host methods. RestoreKV+ + ATTIC adds the restore tokens and LoRA adapter that the RestoreKV authors released for Qwen3-8B and Llama-3.1-8B, unchanged, to ATTIC’s selection. They were trained on KVzip+ selections; retraining them on ours for 1000 steps does not help (83.6 vs. 84.3, Qwen3-8B RULER-4K development split).

G The ATTIC Algorithm

Algorithm 1 ATTIC: query-agnostic, one-shot compression of a document’s KV cache

Require: document C ; model f ; keep ratio r ; host scoring rule ϕ (default: Eq. (2)); optional restore tokens $\mathcal{T}$

Ensure: retained cache KV_S

```

1: Prefill  $C$  to obtain the entries  $\mathcal{E}$  and KeyDiff scores  $\kappa$ 
2:  $Q \leftarrow f(C \oplus \text{QAPROMPT})$  ▷ 6 quoted QA pairs
3:  $s \leftarrow 0$ 
4: for each chunk  $C_j$  of  $C$  do
5:    $\mathcal{A}_j \leftarrow \text{top-}\lfloor r|C_j| \rfloor \text{ positions of } C_j \text{ by } \kappa$  ▷ what
6:    $m_j \leftarrow \# \text{tokens of the sentences that contain } \mathcal{A}_j$  ▷ how much
7:    $R_j \leftarrow \text{top-}m_j \text{ positions of } C_j \text{ by } \kappa, \text{ in order}$ 
8:    $s \leftarrow \max(s, \phi(R_j; C_j), \phi(H \oplus Q; C_j))$ 
9: end for
10:  $\mathcal{S} \leftarrow \text{Top}_r(s)$  over all layers and heads
11: return  $KV_S$ , with  $\mathcal{T}$  attached if given
```
